

Tarea-Examen 1

Complejidad Computacional

Berriel Flores José M.

Junio 2026

Ejercicio 1:

1.a) Entrada w , Salida ww

Procedimiento: La máquina recorre w de izquierda a derecha, copiando símbolo por símbolo al final de la cinta. Una vez que un símbolo es copiado, la máquina coloca una marca para no volverlo a copiar.

Alfabeto: Sea $\Sigma = \{0, 1\}$, $\Lambda = \{0, 1, \hat{0}, \hat{1}, B\}$, donde $\hat{0}$ y $\hat{1}$ son versiones *marcadas* de los símbolos para indicar cuál ya fue previamente copiado.

Conjunto de estados: $Q = \{q_0, q_1, q_2, q_3, q_4, q_f\}$

Función de transición:

1. Recorrer toda la cinta hasta el primer B y colocar $\$$
2. En q_0 : escanea el primer símbolo no marcado de w . Si es 0, lo marca como $\hat{0}$ y pasa a q_2 ; si es 1, lo marca como $\hat{1}$ y va a q_3 ; si es $\$$, pasa a q_f y termina.
3. En $q_{2,3}$: se desplaza a la derecha sobre todos los símbolos (incluyendo marcados y $\$$) hasta encontrar el primer B (final de la cinta), escribe $s \in \{0, 1\}$ según sea el caso y pasa a q_4 .
4. En q_4 : se desplaza a la izquierda hasta encontrar el último símbolo marcado ($\hat{0}$ o $\hat{1}$), luego avanza una celda a la derecha, y regresa a q_0 .

Tabla de transiciones:

Estado	Lee	Escribe	D	Nuevo estado
q_0	0	$\hat{0}$	R	q_2
q_0	1	$\hat{1}$	R	q_3
q_0	$\hat{0}$	$\hat{0}$	R	q_0
q_0	$\hat{1}$	$\hat{1}$	R	q_0
q_0	\$	\$	—	q_f
q_2	0	0	R	q_2
q_2	1	1	R	q_2
q_2	$\hat{0}$	$\hat{0}$	R	q_2
q_2	$\hat{1}$	$\hat{1}$	R	q_2
q_2	B	0	L	q_4
q_3	0	0	R	q_3
q_3	1	1	R	q_3
q_3	$\hat{0}$	$\hat{0}$	R	q_3
q_3	$\hat{1}$	$\hat{1}$	R	q_3
q_3	B	1	L	q_4
q_4	0	0	L	q_4
q_4	1	1	L	q_4
q_4	$\hat{0}$	$\hat{0}$	R	q_0
q_4	$\hat{1}$	$\hat{1}$	R	q_0

Complejidad: Sea $n = |w|$. Para copiar cada uno de los n símbolos, el cabezal recorre aproximadamente n celdas y se repite n veces.

$$T(n) = O(n^2)$$

1.b) Entrada $x\#y$, Salida 1 si $x = y$, 0 si $x \neq y$

Procedimiento: La máquina compara símbolo a símbolo las cadenas x y y , usando marcas para registrar el progreso. Suponga que las dos cadenas están separadas por $\#$.

Alfabeto: $\Sigma = \{0, 1, \#\}$, $\Lambda = \{0, 1, \hat{0}, \hat{1}, \#, B\}$.

Estados: $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_f\}$

Descripción:

1. En q_0 : lee el primer símbolo sin marcar de x . Si es B (toda x procesada), verifica que y también esté completamente marcada; si sí, escribe 1 y acepta (q_f); si no, escribe 0 y rechaza. Si lee 0, lo marca ($\hat{0}$) y pasa a q_1 ; si lee 1, lo marca y pasa a q_2 .
2. En $q_{1,2}$: avanza hacia la derecha cruzando $\#$ y los símbolos de y ya marcados, hasta encontrar el primer símbolo sin marcar de y . Si dicho símbolo es 0 o 1, lo marca y regresa a q_0 (mediante q_5); si es $\hat{0}$ o $\hat{1}$, escribe 0 y pasa a q_7 .
3. q_5 : retrocede hacia la izquierda hasta el último símbolo marcado de x , avanza una posición y regresa a q_0 .
4. q_7 : Estado de rechazo

Complejidad: Sea $n = |x| + |y| + 1$. Para cada uno de los $|x|$ símbolos se recorre la cinta ida y vuelta, lo que toma $O(n)$ pasos.

$$T(n) = O(n^2)$$

1.c) Entrada: número w y conjunto A , Salida: 1 si $w \in A$, 0 si $w \notin A$

Suponga que cada número ocupa exactamente k celdas, y están separados por blancos (B). El primer bloque es w , los siguientes conforman A . Entre w y A hay un símbolo $\#$.

Procedimiento: La máquina compara el bloque w con cada elemento de A secuencialmente (celda a celda, de manera similar al inciso 1.b). Si encuentra igualdad, escribe 1 y acepta; si agota todos los elementos sin alcanzar la igualdad, escribe 0.

Alfabeto: $\Sigma = \{0, 1, B\}$, $\Lambda = \{0, 1, \hat{0}, \hat{1}, B\}$.

Estados: $Q = \{q_0, q_a, q_b, q_c, q_d, q_e, q_f, q_F\}$ **Descripción:**

1. En q_0 : EL cabezal se encuentra al inicio de w : Si lee 0, lo marca como $\hat{0}$ y pasa a q_a . Si lee 1, lo marca como $\hat{1}$ y pasa a q_b . Si lee $\#$, pasa a q_e
2. En $q_{a,b}$: posiciona el cabezal al inicio del siguiente elemento de A (con los símbolos no marcados) y lo compara con w (regresando al inicio de w cada vez). Si lee 0 o 1, lo marca y pasa a q_d . Si lee el símbolo opuesto a 0 o a 1, hay una discrepancia y pasa a q_c . Si hay $\#$ o B , pasa a q_f .
3. q_d : El cabezal retrocede a la izquierda hasta encontrar el último símbolo marcado de w , avanza una celda y pasa a q_0 .
4. q_c : El cabezal borra todas las marcas del elemento actual de A y avanza al siguiente B y coloca $\#$, después retrocede hasta el primer símbolo marcado de w y borra todas las marcas. Regresa a q_0
5. En q_e : Escribe 0 en la cinta y pasa a q_F .
6. En q_f : Escribe 1 en la cinta y pasa a q_F .

Complejidad: Sea n el tamaño total de la cadena de entrada. Si $|A| = m$ y cada número tiene k dígitos, entonces $n = k(m + 1) + m \approx k(m + 1)$. Por cada elemento de A se recorre la cinta ida y vuelta, lo que toma $O(n)$; y hay $O(n/k)$ elementos, entonces:

$$T(n) = O(n^2/k) = O(n^2)$$

Ejercicio 2:

Entrada: La matriz de adyacencia codificada en la cinta como una secuencia de renglones de la forma $\langle \text{etiqueta } a_{1j} \ a_{2j} \cdots a_{nj} \rangle$.

Salida: Los renglones correspondientes a los nodos en la rama recorrida.

Idea general: La máquina inicia desde el primer renglón (primer nodo) y, en cada paso, lee la fila del nodo actual para encontrar el primer vecino no visitado (es decir, la primera

entrada con valor 1 que aún no ha sido marcada). La máquina sigue esa rama hasta que no hay más vecinos disponibles o se llega a una hoja.

Alfabeto: $\Sigma = \{0, 1, A, B\} \cup \mathcal{V}$, donde $\mathcal{V}$ es el conjunto de etiquetas de vértices. Λ se define como $\Sigma \cup \{s'\}$ para cada $s \in \Sigma$.

Estados:

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_f\}$$

Función de transición:

1. q_0 : El cabezal está al inicio de la cinta. Se verifica la etiqueta del nodo actual en una zona de trabajo de la cinta. Se va a q_1 .
2. q_1 : Se recorre la fila del nodo actual buscando el primer 1 (primer vecino). Si se encuentra un 1, se reemplaza por B , se registra el índice de la entrada (corresponde a la columna en la matriz de adyacencia del vecino) y se va a q_2 . Si toda la fila es 0, se ha llegado a una hoja y se va a q_f y q_3 .
3. q_2 : Con base en el índice j del vecino encontrado, se localiza el renglón correspondiente a ese nodo. Para ello, el cabezal recorre los renglones restantes y en cada entrada verifica si se trata de una etiqueta o un elemento de $\{0, 1, B\}$. Cuando encuentra la etiqueta del vecino j la reemplaza por A . Pasa a q_1 .
4. q_3 : El cabezal recorre toda la cinta y todas las celdas de los renglones cuya etiqueta esté marcada con A las marca con B . Se pasa a q_f .
5. q_f : Estado final. La cinta contiene los renglones de los nodos en la rama recorrida.

Nota: Se sigue siempre el *primer* vecino (columna más a la izquierda con valor 1), lo que garantiza que se recorre exactamente una rama sin retroceder.

Complejidad: Sea k el tamaño de la cadena de entrada. La matriz de adyacencia de un grafo con n nodos tiene n^2 entradas, por lo que $k = O(n^2)$ (más etiquetas). Cada rama tiene a lo más n nodos y en cada paso el cabezal recorre a lo más k pasos, con lo cual $T(k) = O(n \cdot k) = O(\sqrt{k} \cdot k) = O(k^{3/2})$. El barrido final suma $O(k)$ pasos. Por lo tanto, el costo total es:

$$T(x) = O(x^{3/2})$$

Ejercicio 3:

En todos los incisos trabajamos con números en representación unaria: el número n se representa como n repeticiones del símbolo 1, i.e., 1^n . Para la representación binaria se ofrecen las complejidades correspondientes en la nota al final.

3.a) Suma de dos números naturales

Entrada: $1^m \# 1^k$ **Salida:** 1^{m+k}

Procedimiento: Reemplazar el símbolo separador $\#$ por 1 y eliminar el último 1 (o equivalentemente, borrar el $\#$ y un 1).

Función de transición:

Estado	Lee	Escribe	Mueve	Nuevo estado
q_0	1	1	R	q_0
q_0	$\#$	1	R	q_1
q_1	1	1	R	q_1
q_1	B	B	L	q_2
q_2	1	B	$-$	q_F

En q_2 se borra el último 1 (para compensar la cantidad de unidades totales: hay un 1 adicional como reemplazo del $\#$ sustituido), y se llega al estado final.

Complejidad: El cabezal recorre toda la cinta una sola vez.

$$\boxed{T(n) = O(n)}, \quad n = m + k + 1$$

3.b) Multiplicación de dos números naturales

Nota: En este caso, se trata de una operación complicada si es realizado por una máquina de Turing estándar. Se recomienda utilizar una máquina de múltiples cintas, sin embargo aquí se presenta la descripción simplificada de una propuesta con una sola cinta.

Entrada: $1^m \# 1^k$ **Salida:** $1^{m \cdot k}$

Procedimiento: Se implementa la multiplicación como sumas repetidas: sumar 1^k exactamente m veces. El resultado de la multiplicación se almacena al final de la cinta.

Descripción:

1. Se marca un 1 de 1^m (indicando que se procesó una copia).
2. Se copia 1^k al final de la zona de resultado.
3. Se repite hasta haber marcado todos los 1's de 1^m .

Complejidad: Cada copia de 1^k requiere $O(n)$ pasos, donde $n \approx m + k + m \cdot k$. Se realizan m copias, y el resultado crece, por lo que el costo total es:

$$\sum_{i=1}^m O(i \cdot k) = O(m^2 \cdot k) \subseteq O(n^3)$$

donde $n = m + k$ es el tamaño de la entrada.

$$\boxed{T(n) = O(n^3)}$$

3.c) Algoritmo de la división

Entrada: $1^k \# 1^m$ **Salida:** q y r tales que $k = q \cdot m + r$, con $0 \leq r < m$.

Procedimiento: Verificar que $k > m$. Si es verdadero: Restar 1^m de 1^k las veces que sea necesario hasta que el residuo r en la cinta cumpla $r < m$ y contar las veces que se hizo (ese conteo es q).

Descripción:

1. Inicializar contador $q = 0$ (al final de la cinta).
2. Intentar restar m unos de los k unos restantes:
 - Si es posible (quedan tantos o más de m unos): borrar m unos, incrementar q en 1; repetir.
 - Si no es posible (quedan menos de m unos): cuantificar el valor del resto r ; terminar.
3. Escribir q y r en la cinta de salida.

La resta de m unos de los k disponibles toma $O(k + m)$ pasos, y se repite $q = \lfloor k/m \rfloor$ veces. El registro de q toma $\approx \sum_{i=1}^k i$. Sea $n = k + m$ el tamaño de entrada.

Complejidad:

$$T = O(q \cdot n + k^2) = O\left(\frac{k}{m} \cdot n\right) \subseteq O(n^2)$$

$$T(n) = O(n^2)$$

Ejercicio 4:

Queremos mostrar que si L es decidible en tiempo $T(n)$ por M MT determinista, entonces existe M' tal que L es decidible en tiempo $O(T(n)^2)$.

4.a) Extensión del lenguaje de M

Suponga que M' tiene el alfabeto extendido:

$$\Lambda' = \Lambda \cup \{\hat{a} \mid a \in \Lambda\}$$

donde $\hat{a}$ es el símbolo a *marcado*. La marca $\hat{a}$ en una celda indica que el cabezal de M' se encuentra en esa celda, y el símbolo que M detectaría es a .

El estado de M' codifica el estado actual de M junto con el símbolo marcado que lee el cabezal, i.e.:

$$Q' = (Q \times \Lambda) \cup \{q'_0, q'_f\}$$

4.b) Movimiento del cabezal de M'

El cabezal de M' sigue el siguiente patrón de *barrido* para cada paso simulado de M :

1. **Inicio:** el cabezal está en la celda 1.
2. **Recorrido derecho:** el cabezal se desplaza desde la celda 1 hasta la celda $T(n)$, visitando todas las celdas.
3. **Recorrido izquierdo:** el cabezal regresa desde la celda $T(n)$ hasta la celda 1.

Este patrón descrito por el cabezal de M' para cualquier entrada x' es idéntico al de x si $|x'| = |x|$, pues el número de celdas barridas $T(n)$ depende únicamente de $|x| = n$, no del contenido de x . Así, M' es olvidadiza.

Como M decide L en tiempo $T(|x|)$, entonces M realiza a lo más $T(|x|)$ pasos de cómputo. En cada paso, el cabezal de M se desplaza a lo más una celda. Por tanto, comenzando desde la celda 1, el cabezal de M nunca puede estar después de la celda $T(|x|)$. Entonces, al hacer que M' recorra hasta la celda $T(|x|)$, se garantiza que siempre se visita la posición donde se encuentra el cabezal de M , para todo t .

4.c) Simulación de un paso de M por M'

Para simular un paso de M , suponga que la máquina M' realiza un recorrido completo (ida y vuelta):

Cuando M' encuentra la celda marcada $\hat{a}$, su estado está dado por:

- El estado actual de M : $q \in Q$.
- El símbolo leído por M : $a \in \Lambda$.

Es decir, M' entra al estado $(q, a) \in Q \times \Lambda$.

A partir de (q, a) , M' se define con la función de transición de M : $\delta(q, a) = (q', b, D)$, donde $q' \in Q$, b es el símbolo a escribir y $D \in \{L, R\}$.

Durante el recorrido (en que M' encontró $\hat{a}$):

- Reemplaza $\hat{a}$ por b .
- Cuando M' llega a la celda adyacente en dirección D , reemplaza el símbolo c que ahí se encuentre por $\hat{c}$ (coloca la marca).
- Actualiza el estado de M' al nuevo estado q' de M .

4.d) Análisis de complejidad de M'

Como M realiza a lo más $T(n)$ pasos y en cada paso el cabezal se mueve a lo más una celda, la distancia máxima desde la celda inicial es $T(n)$.

Para simular cada uno de los $T(n)$ pasos de M , la máquina M' realiza un recorrido completo de la cinta de longitud $T(n)$ (ida y vuelta). Por tanto, por cada paso de M , M' realiza:

$$2 \cdot T(n) = O(T(n)) \text{ pasos.}$$

4.e) Conclusión:

El tiempo total de M' es el número de pasos de M multiplicado por el costo de simular cada paso:

$$T_0(n) = T(n) \cdot O(T(n)) = O(T(n)^2).$$

Por lo tanto, M' es una máquina de Turing **olvidadiza** de orden $O(T(n)^2)$. ■

Ejercicio 5:

Por definición, un lenguaje L pertenece a la clase P sii $\exists M$ máquina de Turing determinista que decide L en tiempo polinomial.

5.a) $L = \{x \in \{0, 1\}^* \mid x \text{ termina en } 11\}$

Algoritmo: Recorrer la cadena de entrada de izquierda a derecha hasta el final, retroceder dos celdas de la cinta y verificar si los símbolos son 11.

Pasos:

1. Mover el cabezal hasta el primer espacio en blanco: $O(n)$ pasos.
2. Retroceder una celda y verificar que sea 1: $O(1)$ pasos.
3. Repetir el paso anterior: $O(1)$ pasos.
4. Aceptar o rechazar.

Tiempo total: $O(n)$, que es polinomial. Por tanto, $L \in P$. ■

5.b) $L = \{0^n 1^n \mid n \geq 0\}$

Algoritmo: Recorrer toda la cadena de entrada hasta el primer 1 y marcarlo. Retroceder hasta el primer 0 y marcarlo. Repetir hasta agotar los 0 o los 1 y si uno se agota y el otro no, se rechaza. Si al agotar un símbolo el otro también lo hace al siguiente paso, se acepta.

Pasos:

1. Recorrer toda la cinta hasta el primer 1 y marcarlo: $O(n)$ pasos.
2. Retroceder hasta el primer 0 no marcado y marcarlo: $2i$ pasos $i \in \{1, \dots, n\}$: $O(n^2 - n)$
3. Repetir hasta que no queden 0's o 1's sin marcar.
4. Aceptar si en cada iteración se marcó exactamente un 0 y un 1, y ambos lados se agotan simultáneamente; rechazar en otro caso.

Tiempo: Hay n iteraciones. Cada iteración requiere recorrer la cinta: $O(2 \cdot i)$ pasos $\forall i = 1, \dots, n$.

Tiempo total: $O(n^2)$, polinomial. Por tanto, $L \in P$. ■

5.c) $L = \{\langle G, u, v \rangle \mid G \text{ es una gráfica y existe un camino de } u \text{ a } v\}$

Algoritmo: Usar Búsqueda en Anchura (BFS) (se descarta Búsqueda en Profundidad porque no es completo) desde u :

1. Representar la gráfica mediante su matriz de adyacencia (dada en la entrada).
2. Ejecutar BFS desde u , marcando los nodos visitados.
3. Si v es alcanzado, se acepta; rechazar si BFS termina sin visitar v .

Complejidad del algoritmo: BFS sobre una gráfica con $|V|$ vértices y $|E|$ aristas toma tiempo $O(|V| + |E|)$. Dado que $|E| \leq |V|^2$, el tiempo es $O(|V|^2)$.

Codificación de la entrada: La matriz de adyacencia tiene $|V|^2$ entradas, por lo que $n = |\langle G, u, v \rangle| = |V|^2$. Esto implica $|V| = O(\sqrt{n})$, y por tanto:

$$O(|V|^2) = O(n).$$

Tiempo total: $O(n)$, que es polinomial. Por tanto, $L \in \text{P}$. ■